BloodConnect Automated Testing Report

Predictive and Prescriptive Decision Support System for Blood Wastage Reduction and Supply Mapping

Purpose: To test the prototype through unit, integration, and system testing to check whether it processes controlled data and produces outputs correctly and consistently. Report date: 13 October 2026 (Asia/Manila). This report only summarizes tests actually executed locally; it does not claim clinical validity or real-world forecast accuracy.

1. Prototype scope and implemented rules

BloodConnect is a React/Vite frontend with an Express/MySQL backend. It supports donor, hospital, administrator, and super-administrator portals. Workflows include sign-in, donor scheduling, hospital inventory and requests, and administrative inventory, requests, transfers, donor, partner, and analytics management.

The analytics engine uses completed or partially fulfilled requests as historical demand; forecasts seven-day demand from the 30-day demand window; excludes expired inventory; limits near-expiry inventory to consumption possible before expiry; classifies trends using +/-10 percent change; and applies recovery periods of 90 days for whole blood, 14 for platelets, and 28 for plasma. Backend allocation prioritizes critical, urgent, then normal compatible requests.

2. Test design and data isolation

Vitest runs deterministic analytics unit tests. Node's built-in runner checks Express API routing and validation. Playwright runs browser workflows and intercepts every API call with in-memory responses, so it does not contact MySQL or production services. The separate database simulation workflow only permits names ending in _simulation and disables schedulers in simulation mode.

Expected unit-test values were hand-calculated from fixed records, rather than obtained from the functions under test. The existing controlled simulation separately compares repeated complete outputs against a human-reviewable oracle.

3. Executed results

Suite	Actual result	Evidence
Vitest analytics unit tests	PASS - 8/8	Fixed inputs; unit HTML report
Playwright browser workflows	PASS - 6/6	HTML report and screenshots
Simulation API/database integration	PASS - 4/4	Real Express + MySQL simulation
Express API tests	PASS - 8/8	Node test-runner output
Controlled analytics oracle	PASS - 128/128	JSON validation report
Database analytics simulation	PASS - 41/41	Guarded MySQL validation
Analytics edge-case regression	PASS - 19/19	Node test-runner output
Frontend production build	PASS	Vite build output

4. Test register

ID	Description	Expected result	Actual
BC-UNIT-001	Inventory coverage	20 stock, 30 demand, 7 next-7-day demand, sufficient	PASS
BC-UNIT-002	Expired stock boundary	No usable stock; critical out	PASS
BC-UNIT-003	Near-expiry boundary	10 units but only 3 usable before expiry	PASS
BC-UNIT-004	Usage trend	60 vs 30; +100%; 14 next-7-day; increasing/high/spike	PASS

ID	Description	Expected result	Actual
BC-UNIT-005	Donor availability	2 eligible now plus 2 in horizon	PASS
BC-UNIT-006	Transfer priority	Critical request receives scarce compatible stock first	PASS
BC-UNIT-007	Repeated run	Same output; input remains unchanged	PASS
BC-UNIT-008	Invalid inputs	Safe empty/low-availability outputs	PASS
BC-E2E-001	Donor workflow	Login routes to /dashboard	PASS
BC-E2E-002	Hospital workflow	Inventory route; O+ mock stock renders	PASS
BC-E2E-003	Admin workflow	Admin route; dashboard widget renders	PASS
BC-INT-001	Admin API/database	Controlled predictive and prescriptive responses	PASS
BC-INT-002	Hospital API/database	Hospital data isolation, requests, reports	PASS
BC-INT-003	Donor API/database	Donor profile, donations, eligibility, schedules	PASS
BC-INT-004	Workflow boundaries	Authorization and invalid request validation	PASS

5. Browser evidence

The passing Playwright run generated the following real screenshot evidence using intercepted test-only API responses: BC-E2E-001-donor-dashboard.png, BC-E2E-002-hospital-inventory.png, and BC-E2E-003-admin-dashboard.png. The files are retained in frontend/test-results/evidence. The corresponding Playwright HTML report is in frontend/test-results/e2e-html/index.html. These demonstrate UI workflow execution, not production-data behavior.

6. Reproduction and limitations

From frontend, run: npm.cmd run test:unit:report and npm.cmd run test:e2e. From backend, run: npm.cmd test. Evidence is generated under frontend/.vitest, frontend/test-results, and backend/simulation/output. For isolated MySQL integration testing, use the guarded simulation database commands documented in docs/Automated_Testing_Guide.md.

A passing result establishes conformance to implemented rules under controlled conditions. It does not establish forecast accuracy, clinical effectiveness, or behavior against production records.